Introduction to Artificial Intelligence

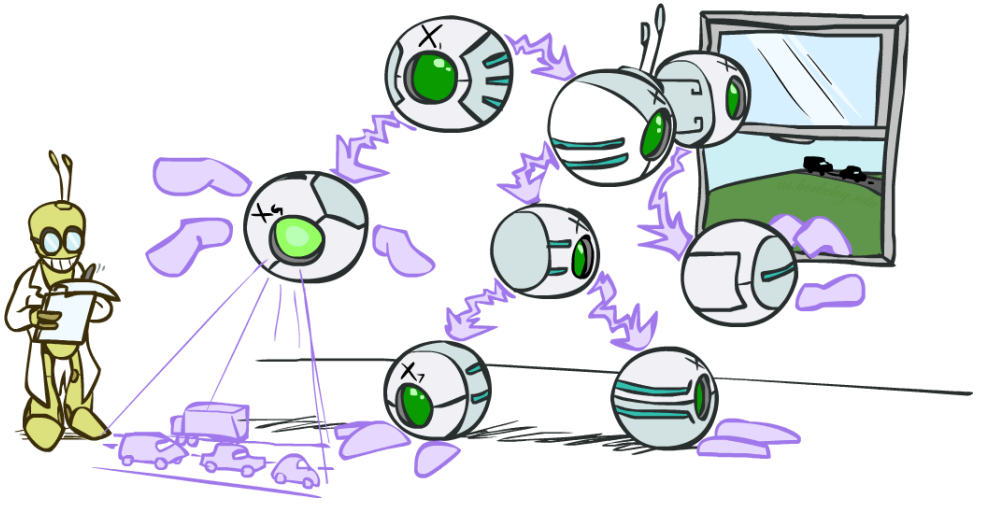
Lecture 5: Probabilistic reasoning

Prof. Gilles Louppe
g.louppe@uliege.be

LIÈGE université

Today

- Bayesian networks
  - Semantics
  - Construction
  - Independence relations
- Inference
- Parameter learning

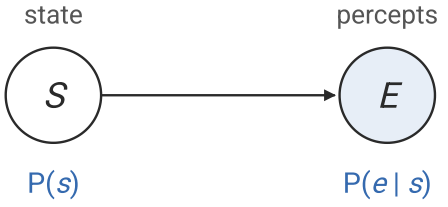


Representing uncertain knowledge

When the environment is only partially observable, the agent must reason about the state s from its percepts e. Its model is the joint distribution

$$P(s, e) = P(s)P(e \mid s),$$

from which it infers the belief $P(s \mid e)$.



When s and e hold many variables, the table of $P(s, e)$ grows exponentially. A **Bayesian network** factors it with conditional independence assumptions.

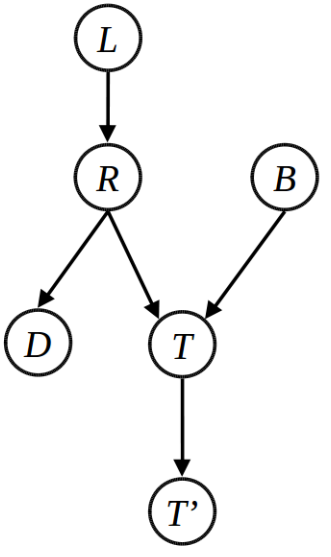
Bayesian networks

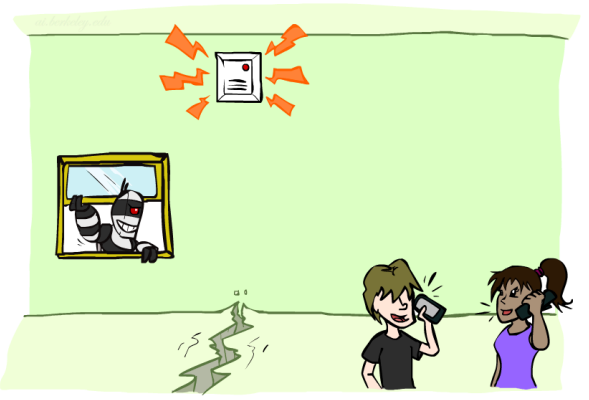
A Bayesian network is a **directed acyclic graph** where

- each **node** corresponds to a random variable;
  - observed, in the percepts $\boldsymbol{e}$, or unobserved, in the state $\boldsymbol{s}$
  - discrete or continuous
- each **edge** is directed and indicates a direct probabilistic dependency between two variables;
- each node X_i is annotated with a **conditional probability distribution**

$$\mathbf{P}(X_i \mid \text{parents}(X_i))$$

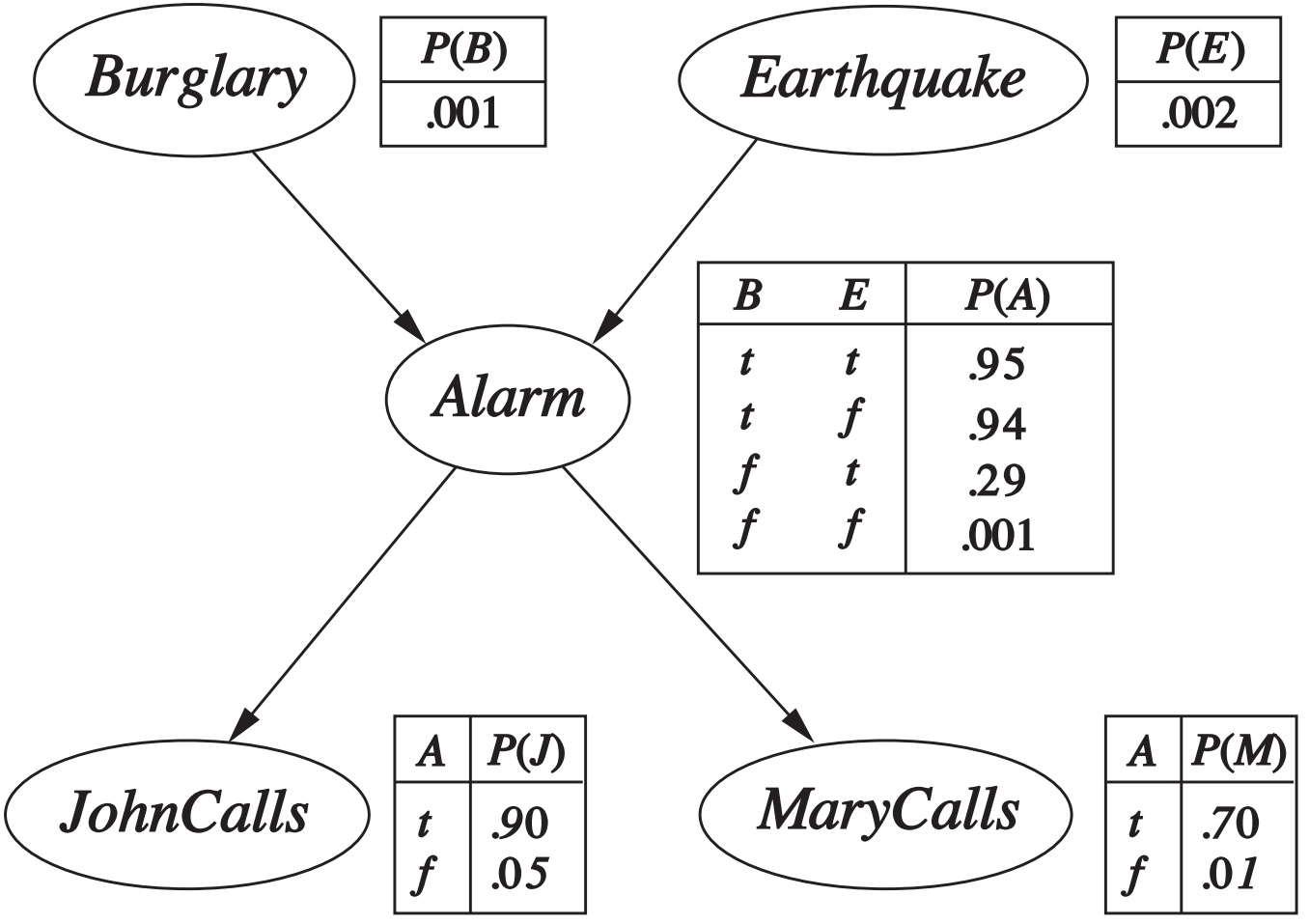
that defines the distribution of X_i given its parents in the network.





Example 1:

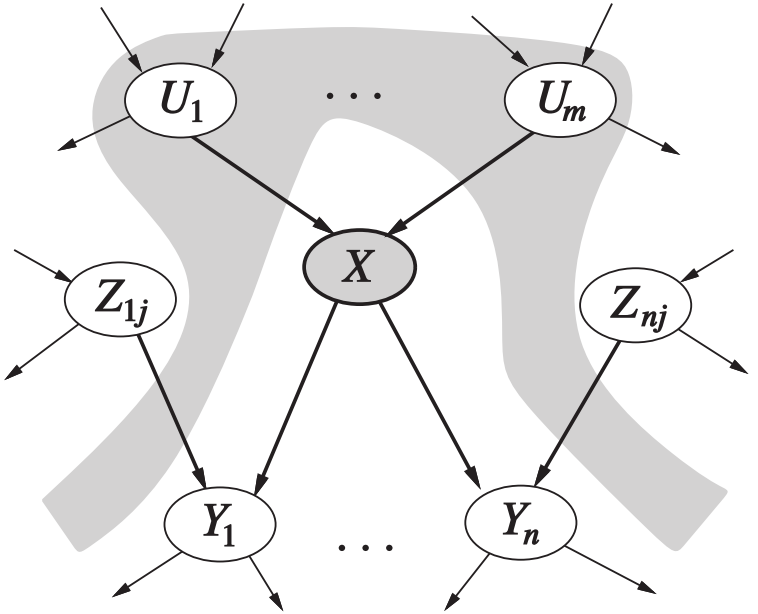
- State variables: **Burglar**, **Earthquake**, **Alarm**.
- Percepts: **JohnCalls**, **MaryCalls**.
- The network topology can be defined from domain knowledge:
  - A burglar can set the alarm off
  - An earthquake can set the alarm off
  - The alarm can cause Mary to call
  - The alarm can cause John to call


Burglary
P(B)
.001
Earthquake
P(E)
.002
Alarm
B E P(A)
t t .95
t f .94
f t .29
f f .001
JohnCalls
A P(J)
t .90
f .05
MaryCalls
A P(M)
t .70
f .01

Semantics

The topology of a network asserts the **local Markov property**: each variable is conditionally independent of its non-descendants, given its parents,

$$X_i \perp \text{non-descendants}(X_i) \mid \text{parents}(X_i).$$



Factorization

A Bayesian network therefore encodes the full joint distribution as a product of local distributions,

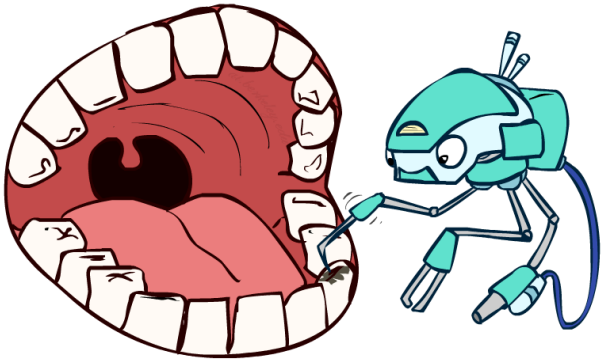
$$P(x_1, ..., x_n) = \prod_{i=1}^{n} P(x_i \mid \mathbf{parents}(X_i)).$$

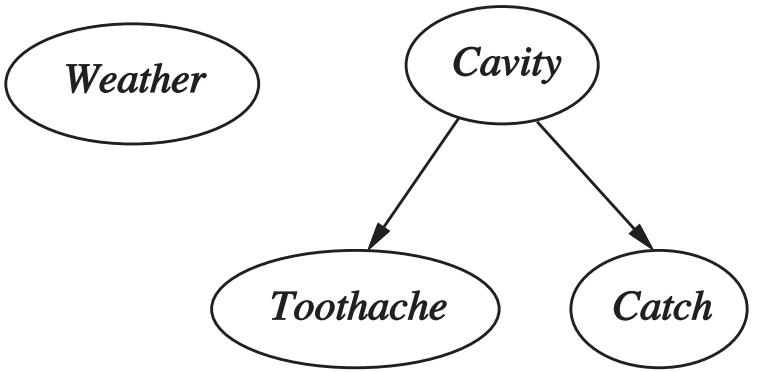
Proof. By the chain rule, $P(x_1, ..., x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, ..., x_{i-1})$. Numbering the variables in a topological order puts $\mathbf{parents}(X_i) \subseteq \{X_1, ..., X_{i-1}\}$ and makes the other predecessors non-descendants of X_i, so the local Markov property gives $P(x_i \mid x_1, ..., x_{i-1}) = P(x_i \mid \mathbf{parents}(X_i))$.

Example 1 (continued):

$$\begin{aligned} P(j, m, a, \neg b, \neg e) &= P(j \mid a) P(m \mid a) P(a \mid \neg b, \neg e) P(\neg b) P(\neg e) \\ &= 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998 \\ &\approx 0.00063 \end{aligned}$$

Example 2:





The dentist's scenario can be modeled as a Bayesian network with four variables, as shown on the right.

By construction, the topology of the network encodes conditional independence assertions. Each variable is independent of its non-descendants given its parents:

- **Weather** is independent of the other variables.
- **Toothache** and **Catch** are conditionally independent given **Cavity**.

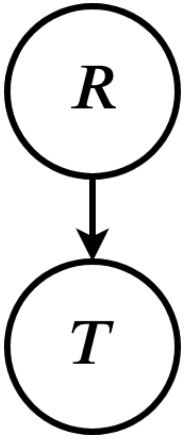






Example 3:

Edges may correspond to causal relations.



$\mathbf{P}(R)$

R	P
r	0.25
¬r	0.75

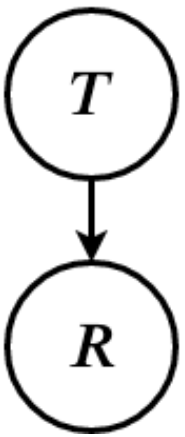
$\mathbf{P}(T \mid R)$

R	T	P
r	t	0.75
r	¬t	0.25
¬r	t	0.5
¬r	¬t	0.5



Example 3 (bis):

... but edges need not be causal!



$\mathbf{P}(T)$

T	P
t	$9/16$
$\neg$t	$7/16$

$\mathbf{P}(R \mid T)$

T	R	P
t	r	$1/3$
t	$\neg$r	$2/3$
$\neg$t	r	$1/7$
$\neg$t	$\neg$r	$6/7$

Construction

Bayesian networks can be constructed in any order, provided that the conditional independence assertions are respected.

function BUILD-NETWORK($order$) **returns** a Bayesian network

  $net \leftarrow$ an empty network

  for each variable X in $order$ **do**

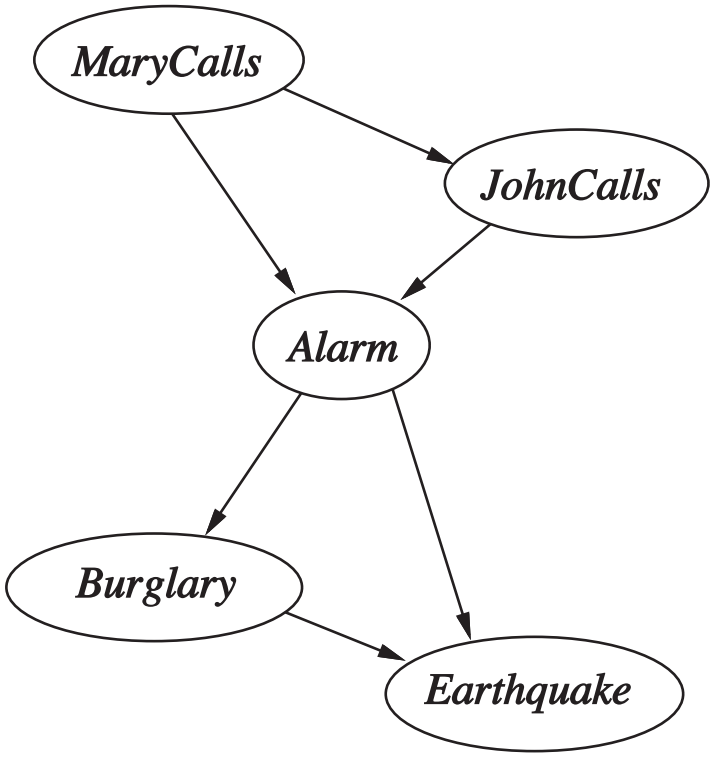
    $parents \leftarrow$ a minimal set of variables before X in $order$

      such that $P(X \mid parents) = P(X \mid$ the variables before $X)$

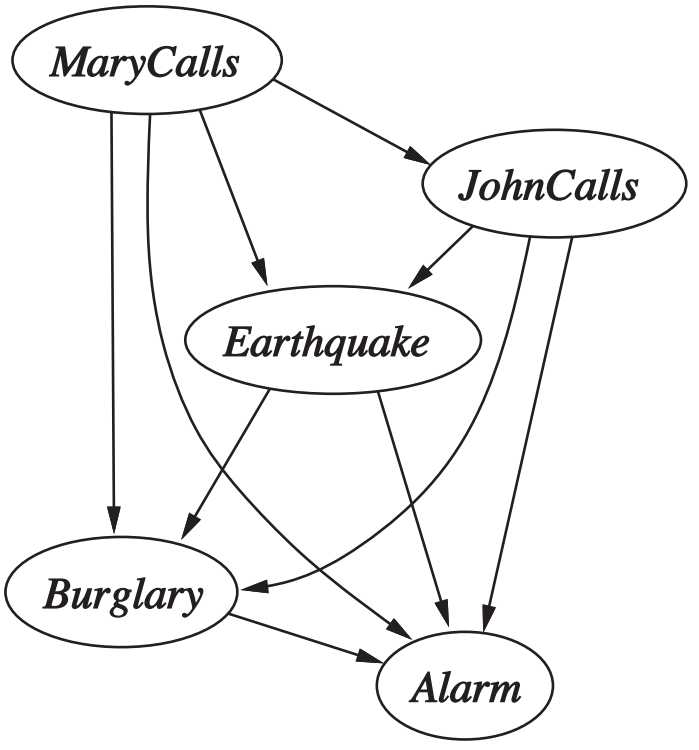
    add X to net, with an edge from each of its $parents$

    write down the CPT of X given its $parents$

  return net



(a)

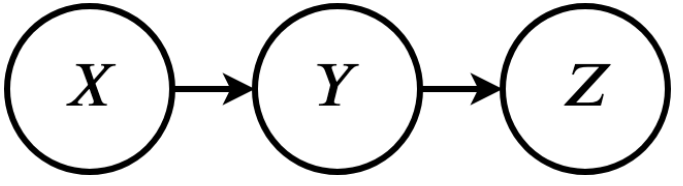


(b)

Do these networks represent the same distribution? Are they as compact?

Independence relations

Since the topology of a Bayesian network encodes conditional independence assertions, it can be used to answer questions about the independence of variables given some evidence.



Example: Are X and Z necessarily independent?

Cascades

Is X independent of Z? No.

Counter-example:

- Low pressure causes rain causes traffic, high pressure causes no rain causes no traffic.
- In numbers:
  - $P(y \mid x) = 1$,
  - $P(z \mid y) = 1$,
  - $P(\neg y \mid \neg x) = 1$,
  - $P(\neg z \mid \neg y) = 1$



X: low pressure, Y: rain, Z: traffic.

$$P(x, y, z) = P(x)P(y \mid x)P(z \mid y)$$

Is X independent of Z, given Y? Yes.

$$
\begin{aligned}
P(z \mid x, y) &= \frac{P(x, y, z)}{P(x, y)} \\
&= \frac{P(x)P(y \mid x)P(z \mid y)}{P(x)P(y \mid x)} \\
&= P(z \mid y)
\end{aligned}
$$

We say that the evidence along the cascade **blocks** the influence.



X: low pressure, Y: rain, Z: traffic.

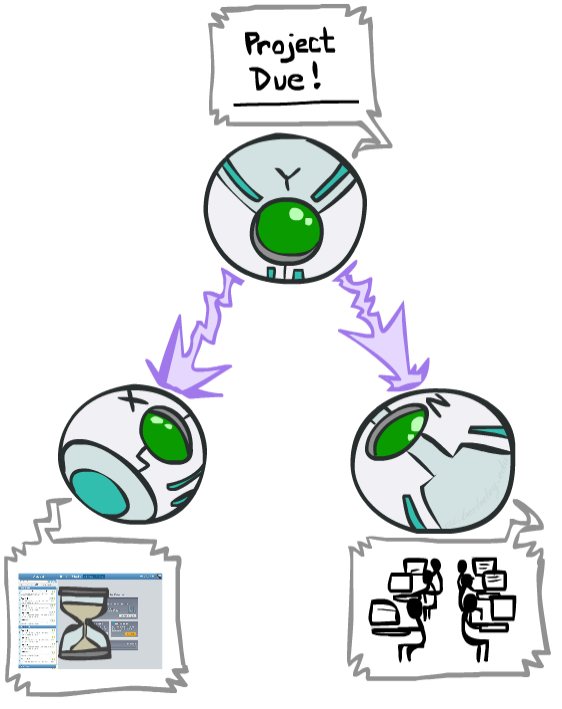
$$P(x, y, z) = P(x)P(y \mid x)P(z \mid y)$$

Common parent

Is X independent of Z? No.

Counter-example:

- Project due causes both forums busy and lab full.
- In numbers:
  - $P(x \mid y) = 1$,
  - $P(\neg x \mid \neg y) = 1$,
  - $P(z \mid y) = 1$,
  - $P(\neg z \mid \neg y) = 1$



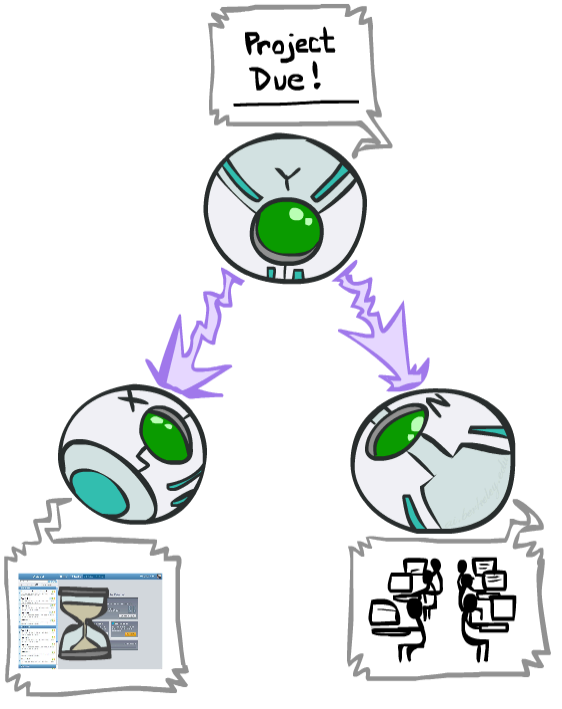
X: forum busy, Y: project due, Z: lab full.

$$P(x, y, z) = P(y)P(x \mid y)P(z \mid y)$$

Is X independent of Z, given Y? Yes

$$
\begin{aligned}
P(z \mid x, y) &= \frac{P(x, y, z)}{P(x, y)} \\
&= \frac{P(y)P(x \mid y)P(z \mid y)}{P(y)P(x \mid y)} \\
&= P(z \mid y)
\end{aligned}
$$

Observing the parent blocks the influence between the children.



X: forum busy, Y: project due, Z: lab full.

$$P(x, y, z) = P(y)P(x \mid y)P(z \mid y)$$

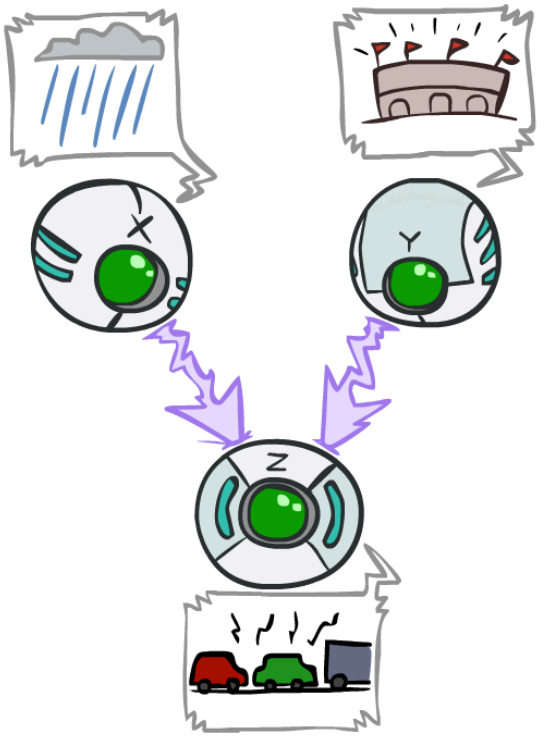
v-structures

Are X and Y independent? Yes.

- The ballgame and the rain cause traffic, but they are not correlated.
- (Prove it!)

Are X and Y independent given Z? No!

- Seeing traffic puts the rain and the ballgame in competition as explanation.
- This is **backwards** from the previous cases. Observing a child node **activates** influence between parents.



X: rain, Y: ballgame, Z: traffic.

$$P(x, y, z) = P(x)P(y)P(z \mid x, y)$$

d-separation

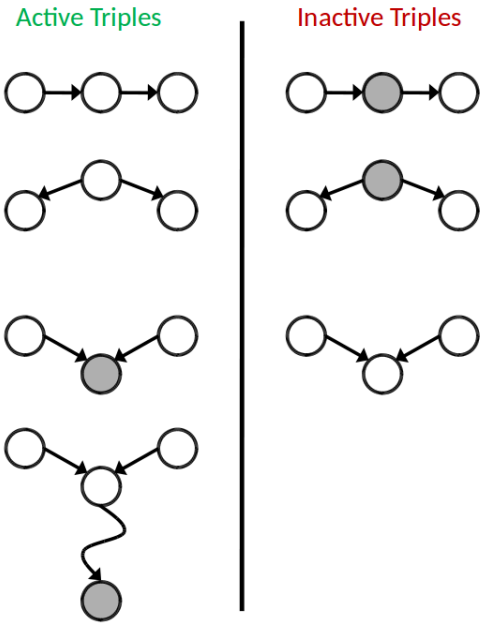
Let us assume a complete Bayesian network. Are X_i and X_j conditionally independent given evidence $Z_1 = z_1, ..., Z_m = z_m$, i.e. does $X_i \perp X_j \mid Z_1, ..., Z_m$ hold?

Consider all (undirected) paths from X_i to X_j:

- If one or more active path, then independence is not guaranteed.
- Otherwise (i.e., all paths are inactive), then independence is guaranteed.

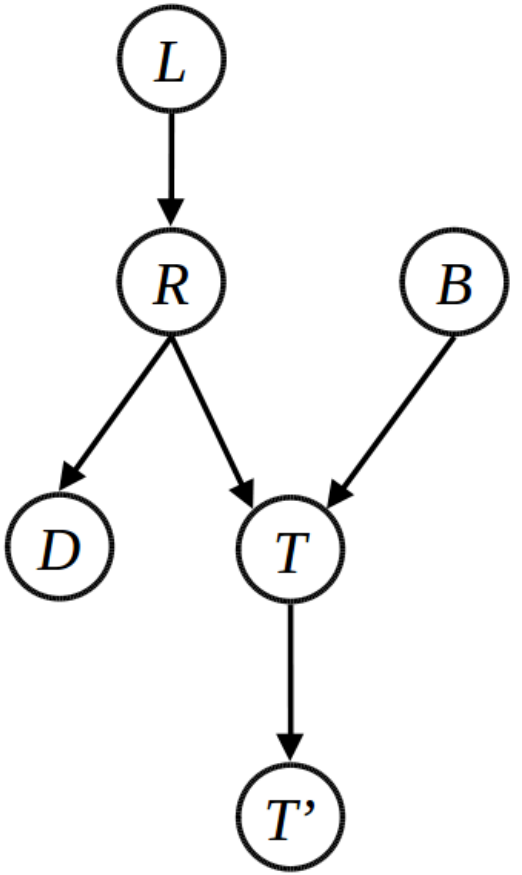
A path is **active** if each triple along the path is active:

- Cascade $A \rightarrow B \rightarrow C$ where B is unobserved (either direction).
- Common parent $A \leftarrow B \rightarrow C$ where B is unobserved.
- v-structure $A \rightarrow B \leftarrow C$ where B or one of its descendants is observed.

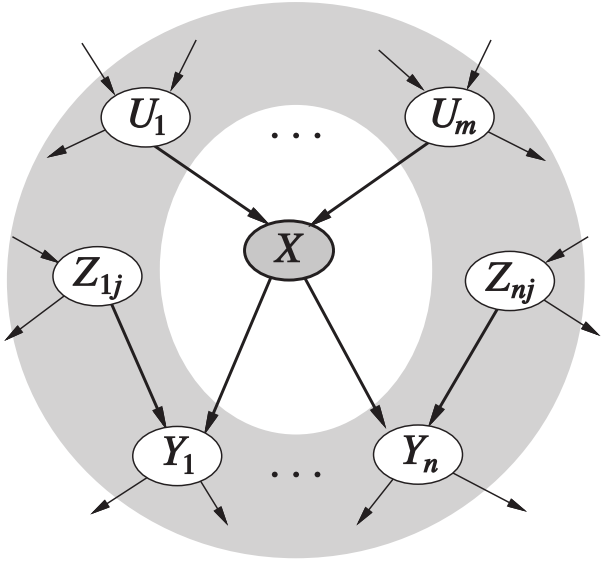


Example:

- $L \perp T' \mid T$?
- $L \perp B$?
- $L \perp B \mid T$?
- $L \perp B \mid T'$?
- $L \perp B \mid T, R$?



Markov blanket



X is conditionally independent of all the other nodes given its **Markov blanket**: its parents, its children, and the other parents of its children.

This follows from d-separation: every path from X to another node is blocked by a node of the blanket. Once its blanket is known, the rest of the network says nothing more about X.

Inference

Probabilistic inference

- **Evidence** variables $\mathbf{E} = \{E_1, ..., E_k\}$, observed as $\mathbf{e} = (e_1, ..., e_k)$.
- **Query** variable Q.
- **Hidden** variables $\mathbf{H} = \{H_1, ..., H_r\}$.
- Together, they are all the variables of the model, $\{Q\} \cup \mathbf{E} \cup \mathbf{H} = \{X_1, ..., X_n\}$.

Inference is the problem of computing the posterior distribution $\mathbf{P}(Q \mid \mathbf{e})$.

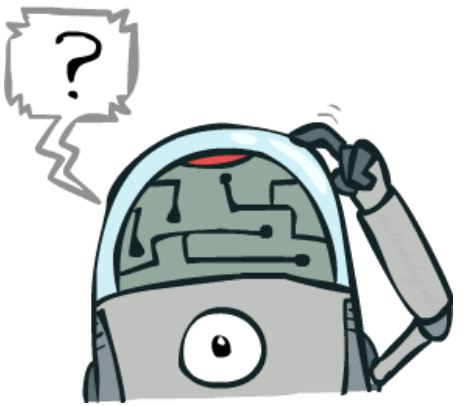
Beyond the posterior of a single variable, the same machinery answers:

Simple queries: $\mathbf{P}(X_i \mid \mathbf{e})$

Conjunctive queries: $\mathbf{P}(X_i, X_j \mid \mathbf{e}) = \mathbf{P}(X_i \mid \mathbf{e})\mathbf{P}(X_j \mid X_i, \mathbf{e})$

Most likely explanation: $\mathbf{arg\,max}_q \, P(q \mid \mathbf{e})$

Optimal decisions: $\mathbf{arg\,max}_a \sum_{s'} P(s' \mid \mathbf{e}, a)V(s')$ (Lectures 8 and 9)



Inference by enumeration

Starting from the joint distribution $\mathbf{P}(Q, \mathbf{E}, \mathbf{H})$, select the entries consistent with the evidence, sum the hidden variables out, and normalize:

$$\mathbf{P}(Q \mid \mathbf{e}) = \frac{1}{Z} \sum_{\mathbf{h}} \mathbf{P}(Q, \mathbf{h}, \mathbf{e}), \qquad Z = \sum_{q} \sum_{\mathbf{h}} P(q, \mathbf{h}, \mathbf{e}).$$

The normalization constant $1/Z$ is also written α.

function ENUMERATION-ASK(Q, e, P) **returns** a distribution over Q

$b \leftarrow$ an empty table over the values of Q

for each value q of Q **do**

$b[q] \leftarrow$ the sum of $P(q, h, e)$ over the assignments h

$Z \leftarrow$ the sum of $b[q]$ over the values q

return b / Z

Example:

- $\mathbf{P}(W)$?
- $\mathbf{P}(W \mid \text{winter})$?
- $\mathbf{P}(W \mid \text{winter}, \text{hot})$?

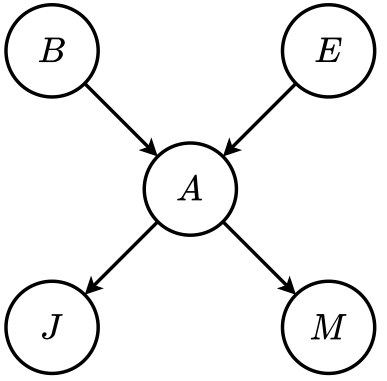
S	T	W	P
summer	hot	sun	0.3
summer	hot	rain	0.05
summer	cold	sun	0.1
summer	cold	rain	0.05
winter	hot	sun	0.1
winter	hot	rain	0.05
winter	cold	sun	0.15
winter	cold	rain	0.2

Inference by enumeration can be used to answer probabilistic queries for **discrete variables** (i.e., with a finite number of values).

However, enumeration **does not scale**! For a domain described by n variables taking at most d values:

- space complexity: $O(d^n)$;
- time complexity: $O(d^n)$.

> Can we do better by exploiting the structure of the network?

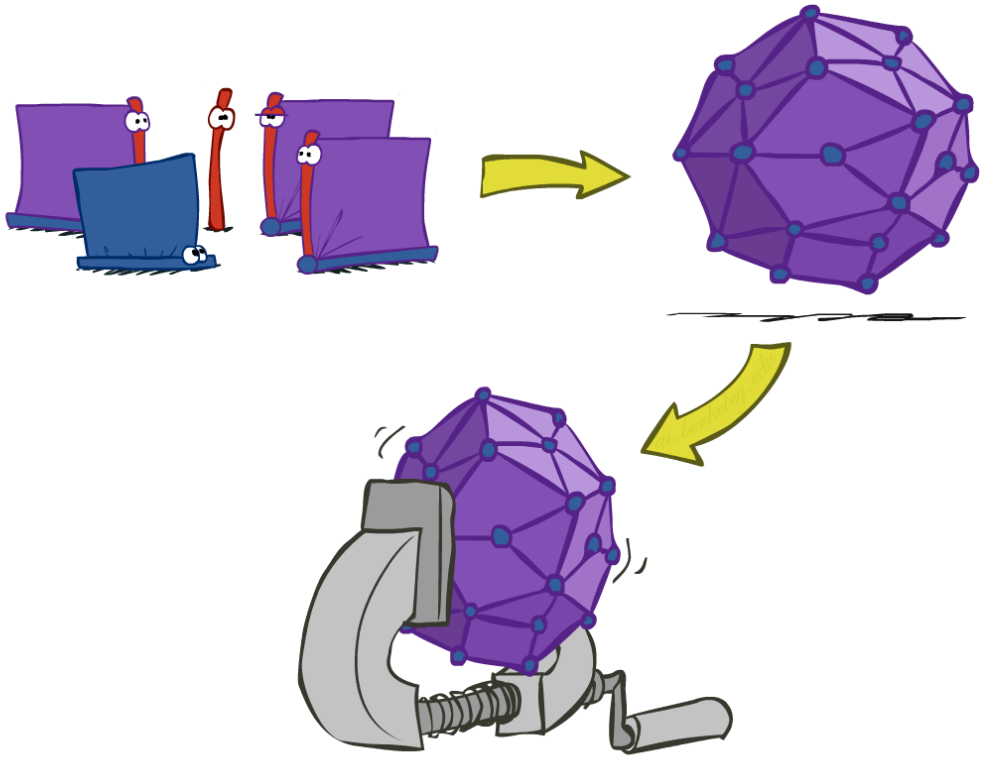


Yes, in part. Consider the alarm network and the query $\mathbf{P}(B \mid j, m)$. We have

$$\begin{aligned}\mathbf{P}(B \mid j, m) &= \frac{1}{Z} \sum_e \sum_a \mathbf{P}(B, j, m, e, a) \\ &\propto \sum_e \sum_a \mathbf{P}(B, j, m, e, a).\end{aligned}$$

Using the Bayesian network, the full joint entries can be rewritten as the product of CPT entries

$$\mathbf{P}(B \mid j, m) \propto \sum_e \sum_a \mathbf{P}(B) P(e) \mathbf{P}(a \mid B, e) P(j \mid a) P(m \mid a).$$



Inference by enumeration is slow because the whole joint distribution is joined up before summing out the hidden variables.

Factors that do not depend on the variables in the summations can be factored out, which means that marginalization does not necessarily have to be done at the end, hence saving some computations.

For the alarm network, we have

$$\begin{aligned}\mathbf{P}(B \mid j, m) &\propto \sum_e \sum_a \mathbf{P}(B) P(e) \mathbf{P}(a \mid B, e) P(j \mid a) P(m \mid a) \\ &= \mathbf{P}(B) \sum_e P(e) \sum_a \mathbf{P}(a \mid B, e) P(j \mid a) P(m \mid a).\end{aligned}$$

Evaluated depth-first over the network, this costs $O(n)$ in space and $O(d^n)$ in time. The structure of the network therefore buys space, and some of the products, but not the exponential time.

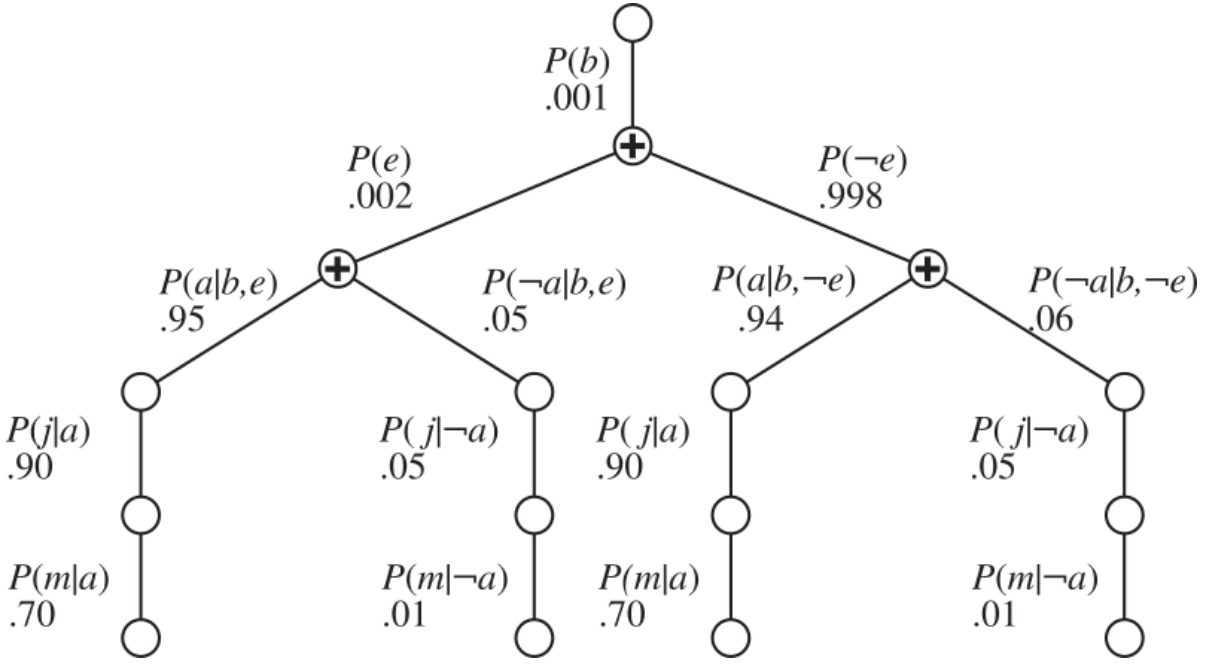
Enumeration over the network

The sums are evaluated depth-first, one variable at a time, multiplying the CPT entries along the way:

function ENUMERATION-ASK(Q, e, bn) **returns** a distribution over Q
  $b \leftarrow$ an empty table over the values of Q
  for each value q of Q **do**
    $b[q] \leftarrow$ ENUMERATE-ALL(the variables of bn, e with $Q = q$)
  return NORMALIZE(b)

function ENUMERATE-ALL($vars$, e) **returns** a number
  if $vars$ is empty **then return** 1
  $V \leftarrow$ the first variable of $vars$, $rest \leftarrow$ the others
  if V has a value v in e **then**
    return $P(v \mid \text{parents}(V)) \times$ ENUMERATE-ALL($rest$, e)
  else return the sum over the values v of V of
    $P(v \mid \text{parents}(V)) \times$ ENUMERATE-ALL($rest$, e with $V = v$)

Evaluation tree for $P(b \mid j, m)$



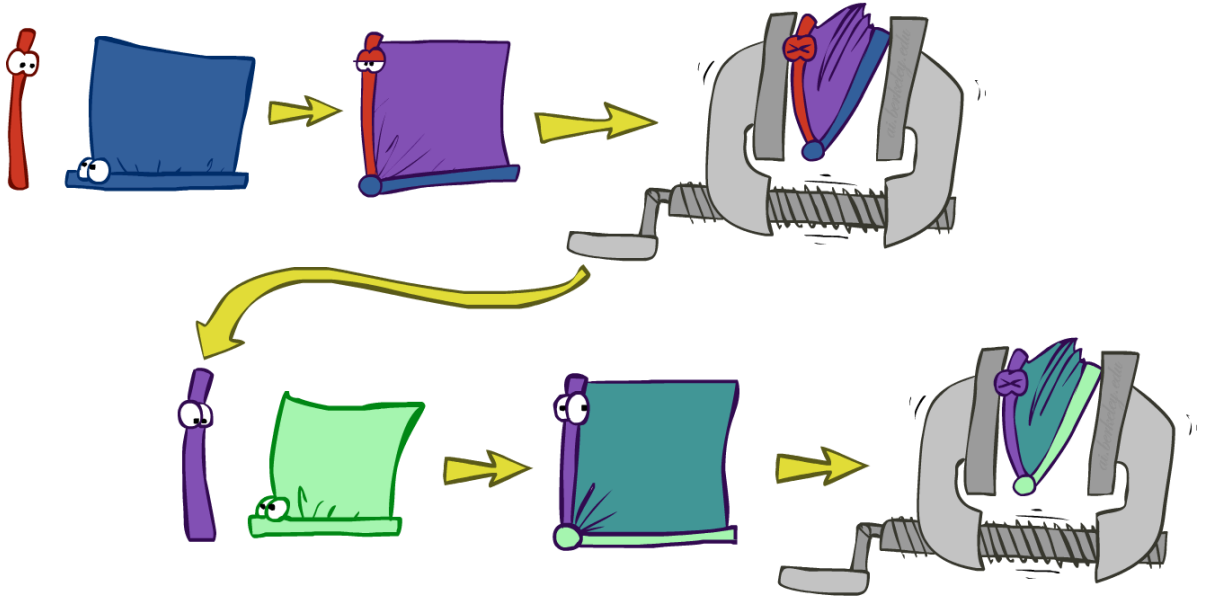
Despite the factorization, inference by enumeration is still **inefficient**. There are repeated computations!

- e.g., $P(j \mid a)P(m \mid a)$ is computed twice, once for e and once for $\neg e$.
- These can be avoided by storing **intermediate results**.

Inference by variable elimination

The **Variable Elimination** algorithm carries out summations right-to-left and stores intermediate factors to avoid recomputations. The algorithm interleaves:

- Joining sub-tables
- Eliminating hidden variables



Variable elimination

For the query $\mathbf{P}(Q \mid \mathbf{e})$:

function ELIMINATION-ASK(Q, e, bn) **returns** a distribution over Q

  $factors \leftarrow$ the CPTs of bn, instantiated by the evidence e

  while a hidden variable is left **do**

    $H \leftarrow$ a hidden variable

    $f \leftarrow$ JOIN(the factors mentioning H)

    $factors \leftarrow$ the factors without H, and ELIMINATE(H, f)

  return NORMALIZE(JOIN($factors$))

The two operations, **join** and **eliminate**, are defined on the next slides.

Factors

- Each **factor** $\mathbf{f}_i$ is a multi-dimensional array indexed by the values of its argument variables. E.g.:

$$\mathbf{f}_4 = \mathbf{f}_4(A) = \begin{pmatrix} P(j \mid a) \\ P(j \mid \neg a) \end{pmatrix} = \begin{pmatrix} 0.90 \\ 0.05 \end{pmatrix}$$

$$\mathbf{f}_4(a) = 0.90$$

$$\mathbf{f}_4(\neg a) = 0.05$$

- Factors are initialized with the CPTs annotating the nodes of the Bayesian network, conditioned on the evidence.

Join

The **pointwise product** $\times$, or **join**, of two factors $\mathbf{f}_1$ and $\mathbf{f}_2$ yields a new factor $\mathbf{f}_3$.

- Exactly like a **database join**!
- The variables of $\mathbf{f}_3$ are the **union** of the variables in $\mathbf{f}_1$ and $\mathbf{f}_2$.
- The elements of $\mathbf{f}_3$ are given by the product of the corresponding elements in $\mathbf{f}_1$ and $\mathbf{f}_2$.

A	B	$\mathbf{f}_1(A,B)$	B	C	$\mathbf{f}_2(B,C)$	A	B	C	$\mathbf{f}_3(A,B,C)$
T	T	.3	T	T	.2	T	T	T	$.3 \times .2 = .06$
T	F	.7	T	F	.8	T	T	F	$.3 \times .8 = .24$
F	T	.9	F	T	.6	T	F	T	$.7 \times .6 = .42$
F	F	.1	F	F	.4	T	F	F	$.7 \times .4 = .28$
						F	T	T	$.9 \times .2 = .18$
						F	T	F	$.9 \times .8 = .72$
						F	F	T	$.1 \times .6 = .06$
						F	F	F	$.1 \times .4 = .04$

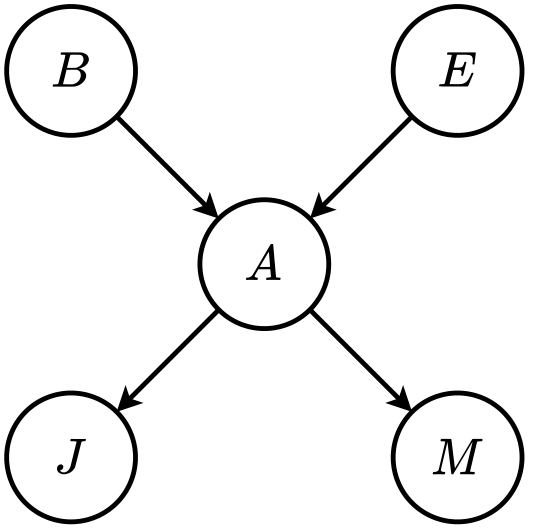
Figure 14.10 Illustrating pointwise multiplication: $\mathbf{f}_1(A,B) \times \mathbf{f}_2(B,C) = \mathbf{f}_3(A,B,C)$.

Elimination

Summing out, or **eliminating**, a variable from a factor is done by adding up the sub-arrays formed by fixing the variable to each of its values in turn.

For example, to sum out A from $\mathbf{f}_3(A, B, C)$, we write:

$$\mathbf{f}(B, C) = \sum_a \mathbf{f}_3(a, B, C) = \mathbf{f}_3(a, B, C) + \mathbf{f}_3(\neg a, B, C)$$
$$= \begin{pmatrix} 0.06 & 0.24 \\ 0.42 & 0.28 \end{pmatrix} + \begin{pmatrix} 0.18 & 0.72 \\ 0.06 & 0.04 \end{pmatrix} = \begin{pmatrix} 0.24 & 0.96 \\ 0.48 & 0.32 \end{pmatrix}$$



Run the variable elimination algorithm for the query $\mathbf{P}(B \mid j, m)$.

Relevance

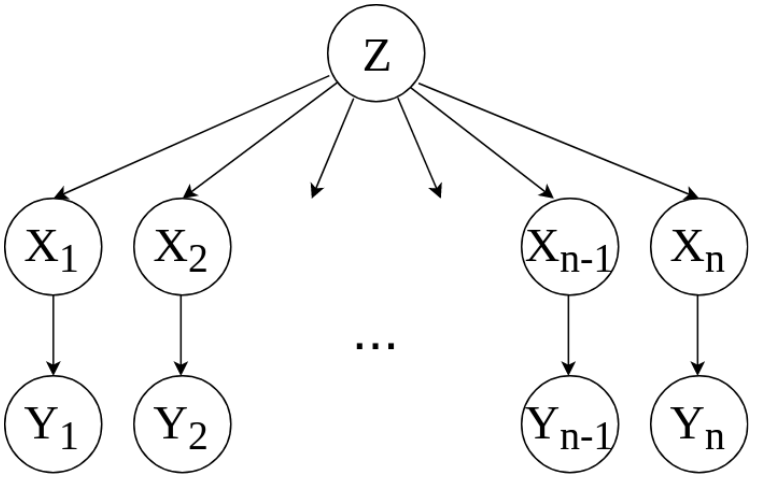
Consider the query $\mathbf{P}(J \mid b)$:

$$\mathbf{P}(J \mid b) \propto P(b) \sum_e P(e) \sum_a P(a \mid b, e) \mathbf{P}(J \mid a) \sum_m P(m \mid a)$$

- $\sum_m P(m \mid a) = 1$, therefore M is **irrelevant** for the query.
- In other words, $\mathbf{P}(J \mid b)$ remains unchanged if we remove M from the network.

Theorem. H is irrelevant for $\mathbf{P}(Q \mid \mathbf{e})$ unless $H \in \mathbf{ancestors}(\{Q\} \cup \mathbf{E})$.

Complexity



Consider the query $\mathbf{P}(X_n \mid y_1, ..., y_n)$.

Work through the two elimination orderings:

- $Z, X_1, ..., X_{n-1}$
- $X_1, ..., X_{n-1}, Z$

What is the size of the maximum factor generated for each of the orderings?

- Answer: 2^{n+1} vs. 2^2 (assuming boolean values)

The computational and space complexity of variable elimination is determined by the largest factor.

- The elimination **ordering** can greatly affect the size of the largest factor.
- The optimal ordering is **NP-hard** to find. There is no known polynomial-time algorithm to find it.

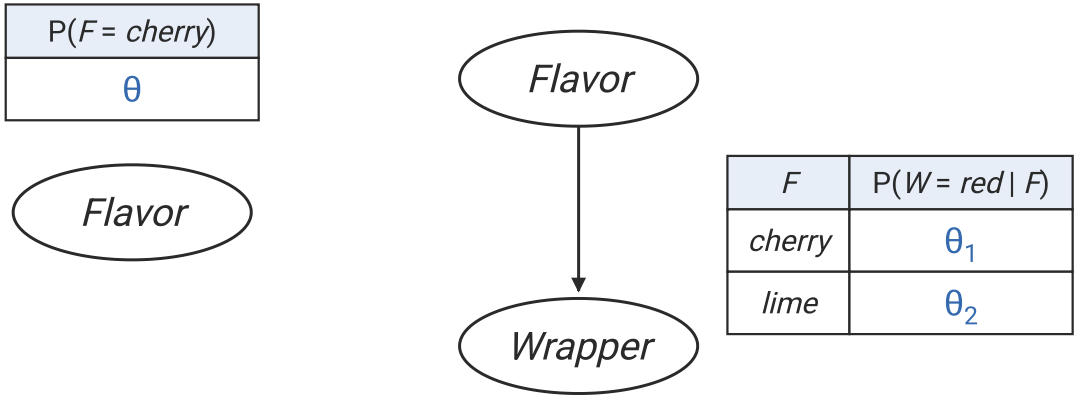
> Exact inference is intractable for most models of interest. For this reason, approximate inference methods are used instead in practice (e.g., sampling, variational inference, or learning from simulations).

Parameter learning

When modeling a domain, we can choose a probabilistic model specified as a Bayesian network. However, specifying the individual probability values is often difficult.

A workaround is to use a **parameterized** family $\mathbf{P}(X \mid \theta)$ (sometimes also noted $\mathbf{P}_\theta(X)$) of models, and **estimate** the parameters θ from data.

> The same principle trains **neural networks** (Lecture 7): a network is a parameterized distribution $p_\theta(y \mid \mathbf{x})$, whose parameters are estimated from data by maximum likelihood.


P(F = cherry)
θ
Flavor
P(F = cherry)
θ
Flavor
Wrapper
F
P(W = red | F)
cherry
θ1
lime
θ2

(a) (b)

Maximum likelihood estimation

Suppose we have a set of N i.i.d. observations $\mathbf{d} = \{x_1, ..., x_N\}$.

The **likelihood** of the parameters θ is the probability of the data given the parameters

$$P(\mathbf{d} \mid \theta) = \prod_{j=1}^{N} P(x_j \mid \theta).$$

The **maximum likelihood estimate** (MLE) θ^* of the parameters is the value of θ that maximizes the likelihood

$$\theta^* = \arg\max_{\theta} P(\mathbf{d} \mid \theta).$$

When possible analytically,

1. Write down the log-likelihood $L(\theta) = \log P(\mathbf{d} \mid \theta)$ of the parameters θ.
2. Write down the derivative $\frac{\partial L}{\partial \theta}$ of the log-likelihood of the parameters θ.
3. Find the parameter values θ^* such that the derivatives are zero (and check whether the Hessian is negative definite).

Case (a)

What is the fraction θ of cherry candies?

Suppose we unwrap N candies, and get c cherries and $l = N - c$ limes. These are i.i.d. observations, therefore

$$P(\mathbf{d} \mid \theta) = \prod_{j=1}^{N} P(x_j \mid \theta) = \theta^c (1-\theta)^l.$$

Maximize this w.r.t. θ, which is easier for the log-likelihood and leads to

$$\begin{aligned} L(\mathbf{d} \mid \theta) &= \log P(\mathbf{d} \mid \theta) = c \log \theta + l \log(1-\theta) \\ \frac{\partial L(\mathbf{d} \mid \theta)}{\partial \theta} &= \frac{c}{\theta} - \frac{l}{1-\theta} = 0. \end{aligned}$$

Hence $\theta = \frac{c}{N}$.

Case (b)

Red and green wrappers depend probabilistically on flavor. E.g., the likelihood for a cherry candy in green wrapper is

$$\begin{aligned}
&P(\text{cherry}, \text{green} \mid \theta, \theta_1, \theta_2) \\
&= P(\text{cherry} \mid \theta, \theta_1, \theta_2) P(\text{green} \mid \text{cherry}, \theta, \theta_1, \theta_2) \\
&= \theta(1-\theta_1).
\end{aligned}$$

The likelihood for the parameters, given N candies, r_c red-wrapped cherries, g_c green-wrapped cherries, etc., is

$$\begin{aligned}
P(\mathbf{d} \mid \theta, \theta_1, \theta_2) &= \theta^c (1-\theta)^l \theta_1^{r_c} (1-\theta_1)^{g_c} \theta_2^{r_l} (1-\theta_2)^{g_l} \\
L &= c \log \theta + l \log(1-\theta) + \\
&\quad r_c \log \theta_1 + g_c \log(1-\theta_1) + \\
&\quad r_l \log \theta_2 + g_l \log(1-\theta_2).
\end{aligned}$$

The derivatives of L yield

$$\begin{aligned}
\frac{\partial L}{\partial \theta} &= \frac{c}{\theta} - \frac{l}{1-\theta} = 0 \Rightarrow \theta = \frac{c}{c+l} \\
\frac{\partial L}{\partial \theta_1} &= \frac{r_c}{\theta_1} - \frac{g_c}{1-\theta_1} = 0 \Rightarrow \theta_1 = \frac{r_c}{r_c+g_c} \\
\frac{\partial L}{\partial \theta_2} &= \frac{r_l}{\theta_2} - \frac{g_l}{1-\theta_2} = 0 \Rightarrow \theta_2 = \frac{r_l}{r_l+g_l}.
\end{aligned}$$

In case (a), if we unwrap 1 candy and get 1 cherry, what is the MLE? How confident are we in this estimate?

Bayesian parameter learning

We can treat parameter learning as a **Bayesian inference** problem:

- Make the parameters θ random variables and treat them as hidden variables.
- Specify a **prior** distribution $\mathbf{P}(\theta)$ over the parameters.
- Then, as data arrives, update our beliefs about the parameters to obtain the **posterior** distribution $\mathbf{P}(\theta \mid \mathbf{d})$.

> How should the network of case (a) be updated?

Case (a)

What is the fraction θ of cherry candies?

We assume a Beta prior

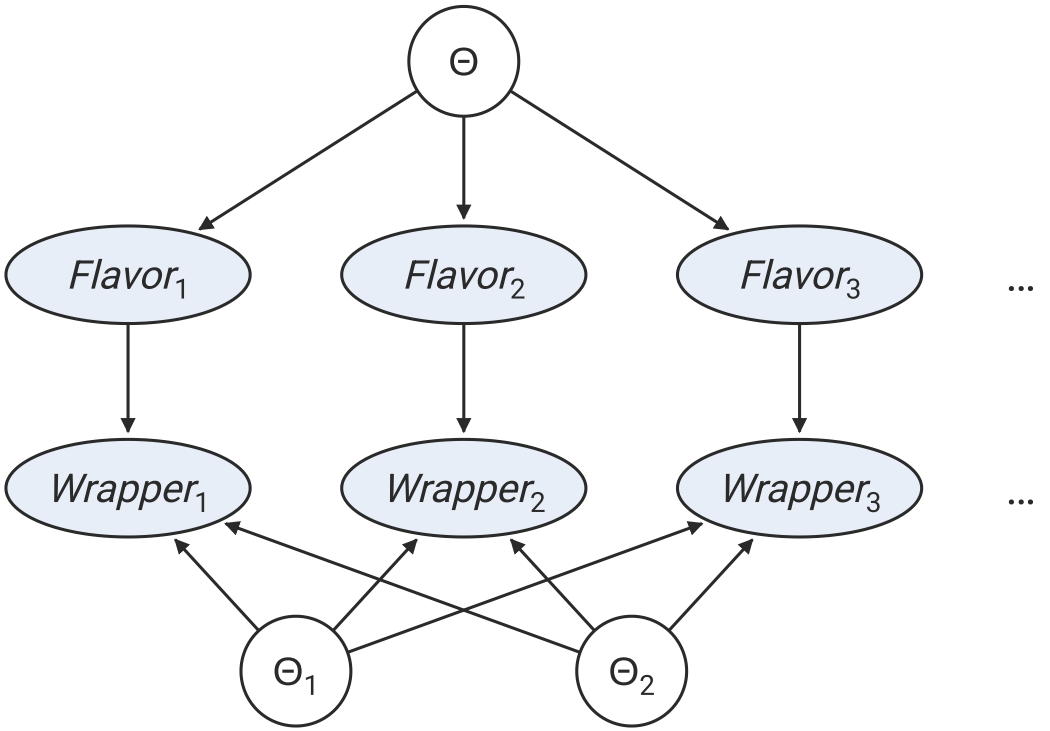
$$P(\theta) = \text{Beta}(\theta \mid a, b) = \frac{1}{Z}\theta^{a-1}(1-\theta)^{b-1}$$

where Z is a normalization constant.

Then, observing a cherry candy yields the posterior

$$\begin{aligned} P(\theta \mid \text{cherry}) &\propto P(\text{cherry} \mid \theta)P(\theta) \\ &= \theta \text{Beta}(\theta \mid a, b) \\ &= \theta\theta^{a-1}(1-\theta)^{b-1} \\ &= \theta^{a}(1-\theta)^{b-1} \\ &= \text{Beta}(\theta \mid a+1, b). \end{aligned}$$

Case (b)


Θ
Flavor1
Flavor2
Flavor3
...
Wrapper1
Wrapper2
Wrapper3
...
Θ1
Θ2

Maximum a posteriori estimation

When the posterior cannot be computed analytically, we can use **maximum a posteriori** (MAP) estimation, which consists in approximating the posterior with the point estimate θ^* that maximizes the posterior distribution, i.e.,

$$\theta^* = \arg\max_{\theta} P(\theta \mid \mathbf{d}) = \arg\max_{\theta} P(\mathbf{d} \mid \theta) P(\theta).$$

Step by step, in code

The candy example, run on `demo/lec5/cherries.ipynb`: draw candies, estimate θ by maximum likelihood, then update a Beta prior and watch the posterior concentrate.

```
uv run jupyter lab demo/lec5/cherries.ipynb
```

Summary

- A **Bayesian network** encodes the joint distribution as a product of local distributions, $P(x_1, ..., x_n) = \prod_{i=1}^{n} P(x_i \mid \mathrm{parents}(X_i))$, with far fewer numbers than the d^n entries of the joint.
- Its topology asserts conditional independences, which **d-separation** reads off the graph: $X_i \perp X_j \mid \mathbf{Z}$ when every path between them is blocked.
- **Inference** computes $\mathbf{P}(Q \mid \mathbf{e}) \propto \sum_{\mathbf{h}} \mathbf{P}(Q, \mathbf{h}, \mathbf{e})$, by enumeration, depth-first and in $O(d^n)$ time, or by **variable elimination**, which reuses factors and costs the size of the largest one.
- Exact inference is intractable in general, and the posterior is then approximated.
- The parameters are estimated from data, by **maximum likelihood**, $\theta^* = \arg\max_\theta P(\mathbf{d} \mid \theta)$, or by a **posterior** $P(\theta \mid \mathbf{d}) \propto P(\mathbf{d} \mid \theta) P(\theta)$, summarized by its maximum.

The end.